SOFTWARE DESIGN DOCUMENT

Jogo Sandbox de Blocos

Referência funcional: Minecraft Java Edition até a versão 1.9

Arquitetura-alvo: Electron · Vite · React · Tailwind CSS · Zustand · React Router · Three.js · Rapier

Tipo de documento: Documento de Design de Software (SDD)

Versão: 1.0 Data: 31/10/2026 Status: Proposta inicial

Documento de referência para planejamento, implementação, testes e evolução de um jogo voxel independente para desktop.

Controle do documento

Campo	Descrição
Produto	Jogo sandbox voxel independente para desktop
Plataforma inicial	Windows, com possibilidade de empacotamento para Linux e macOS
Referência de conteúdo	Conjunto de mecânicas e recursos até Minecraft Java Edition 1.9, inclusive
Stack solicitada	Vite, React, Tailwind CSS, Electron, Zustand e React Router
Dependências técnicas recomendadas	Three.js (renderização 3D), Rapier (física), biblioteca de ruído procedural e sistema de testes
Público-alvo	Jogadores single-player; multiplayer fica fora do escopo inicial
Premissa	Jogo original: nomes, código, texturas, sons, modelos e interface próprios

Como interpretar este SDD

Os requisitos são identificados por códigos estáveis. “Obrigatório” define o escopo mínimo de uma versão jogável; “Opcional” representa expansão planejada. A referência 1.9 é usada como limite de funcionalidades: recursos introduzidos depois dela não fazem parte do escopo-base.

Este documento descreve comportamento e arquitetura, não determina que o projeto copie código, assets ou implementação interna de outro jogo.

1. Visão geral e objetivos

1.1 Propósito

Definir a arquitetura, os subsistemas, os dados, os requisitos e os critérios de qualidade de um jogo sandbox 3D baseado em blocos. O jogador deve poder explorar um mundo procedural, coletar recursos, fabricar itens, construir estruturas, sobreviver a ameaças e progredir entre dimensões.

1.2 Objetivos do produto

- Oferecer um mundo voxel persistente, expansível e explorável sem telas de carregamento a cada região.
- Implementar os ciclos centrais de sobrevivência, mineração, construção, crafting, combate e exploração.
- Permitir modo Criativo com construção livre e inventário de acesso rápido.
- Manter o núcleo do jogo desacoplado da interface React e do processo Electron.
- Permitir salvar e carregar mundos localmente, com recuperação contra gravações interrompidas.
- Manter desempenho previsível em computadores desktop de especificações modestas.

1.3 Princípios de design

- Simulação determinística sempre que possível: mesma seed e parâmetros devem produzir terreno equivalente.
- Dados orientados a conteúdo: blocos, itens, receitas, entidades e biomas devem ser declarativos.
- Separação entre simulação, renderização, interface, persistência e plataforma.
- Atualizações do mundo por regiões/chunks, evitando reconstruir toda a geometria a cada alteração.
- Progressão emergente: objetivos são sugeridos por recursos, receitas, dimensões e desafios, sem exigir uma campanha linear.

2. Escopo funcional

2.1 Incluído na referência até 1.9

Área	Escopo incluído
Mundo	Mundo infinito por geração procedural; seed; chunks; biomas; estruturas; ciclo dia/noite; clima
Blocos	Terreno, líquidos, blocos naturais, minerais, madeira, construção, utilitários e mecanismos
Jogabilidade	Sobrevivência, Criativo, mineração, colocação, coleta, queda, dano, fome, saúde e respawn
Crafting	Bancada 2×2/3×3, fornalha, receitas, ferramentas, armaduras e itens
Entidades	Animais passivos/neutros, monstros hostis, aldeões e entidades especiais
Combate	Armas, armaduras, escudo, mão secundária, arco, flechas especiais, recarga/tempo de ataque
Progressão	Encantamentos, poções, experiência, Nether, End, dragão, cidades do End e elytra
Redstone	Sinais, alavancas, botões, placas, repetidores, comparadores, pistões e circuitos básicos
Interface	HUD, inventário, menus, pausa, configurações, seleção de mundo e telas de morte

2.2 Fora do escopo inicial

- Multiplayer, servidores dedicados, matchmaking, contas online e sincronização em nuvem.
- Marketplace, economia online, conquistas conectadas e conteúdo criado por usuários com distribuição remota.
- Compatibilidade com mundos, saves, mods ou protocolos de Minecraft.
- Recursos introduzidos depois da versão 1.9, como atualização aquática, aldeões modernos, Nether Update e Caves & Cliffs.
- Editor visual completo de mods. A arquitetura deve permitir extensão futura, mas não é requisito da primeira entrega.

2.3 Modos de jogo

Modo	Regras principais
Sobrevivência	Vida, fome, dano, coleta, crafting, inventário limitado, morte e respawn; dificuldade configurável
Criativo	Voo, invulnerabilidade, acesso ao catálogo de blocos/itens e colocação rápida
Espectador (opcional)	Movimentação livre sem interação com blocos ou entidades
Hardcore (opcional)	Sobrevivência com morte permanente ou bloqueio do mundo após morrer

3. Arquitetura tecnológica

3.1 Stack e responsabilidade

Tecnologia	Responsabilidade
Electron	Janela desktop, ciclo de vida, menus nativos, arquivos, diálogos e empacotamento
Vite	Servidor de desenvolvimento, bundling e otimização do renderer
React	Menus, HUD, inventários, telas, overlays e componentes de interface
Tailwind CSS	Sistema visual, layout responsivo e estados da interface
Zustand	Estado de UI, sessão, configurações e estado serializável de alto nível
React Router	Navegação entre menu principal, seleção de mundo, jogo, configurações e créditos
Three.js (recomendado)	Renderização 3D, câmera, materiais, iluminação, shaders e instancing
Rapier (recomendado)	Colisões e física de jogador/entidades; não usar física rígida para cada bloco estático
IndexedDB/arquivos locais	Metadados e persistência; arquivos binários por mundo/chunk são preferíveis para saves grandes

Decisão arquitetural importante: React não deve ser o motor de renderização do mundo 3D. O React desenha a interface; Three.js desenha o mundo em um canvas próprio. O loop de jogo deve executar fora do ciclo de renderização do React.

3.2 Visão em camadas

Camada	Módulos	Regra de dependência
Apresentação	React UI, HUD, menus, Router, Tailwind	Consome APIs/fachadas; não altera chunks diretamente
Aplicação	GameSession, WorldService, InventoryService, SaveService	Orquestra casos de uso
Domínio	World, Chunk, Block, Item, Entity, Recipe, Player, Rules	Não depende de React, Electron ou Three.js
Infraestrutura	ThreeRenderer, PhysicsAdapter, FileStorage, InputAdapter, Audio	Implementa interfaces definidas pelo domínio/aplicação
Plataforma	Electron main/preload, filesystem, janela, permissões	Acesso privilegiado somente pelo processo main

3.3 Processos Electron

- Main process: cria janela, controla permissões, registra IPC mínimo, diálogos e operações de arquivo.
- Preload: expõe uma API tipada e restrita via contextBridge; nunca expõe Node.js inteiro ao renderer.
- Renderer: aplicação React/Vite, canvas Three.js e lógica não privilegiada.
- Ativar contextIsolation, sandbox e nodeIntegration=false; validar todos os argumentos recebidos por IPC.
- Não carregar conteúdo remoto arbitrário nem executar scripts externos.

3.4 Loop de jogo

Usar um loop fixo para simulação e renderização com delta limitado. A simulação deve rodar em passos constantes (por exemplo, 20 ticks por segundo como ponto de partida), enquanto a renderização acompanha a taxa de atualização do monitor. Interpolação visual reduz tremores. Operações caras, como geração de chunks e meshing, devem ser executadas em Web Workers quando viável.

```
updateInput → fixedUpdate (simulação) → streamChunks → updateMeshes → renderFrame → updateUI
snapshots
```

4. Subsistemas do jogo

4.1 Mundo e chunks

- Dividir o mundo em chunks de dimensões configuráveis; referência inicial: 16×16 blocos no plano horizontal e altura de mundo definida pelo perfil.
- Cada chunk armazena IDs de blocos, metadados, luz, entidades locais e estado de modificação.
- Carregar e descarregar chunks por distância da câmera/jogador, com prioridades por proximidade e direção de movimento.
- Gerar terreno em etapas: altura, bioma, cavernas, líquidos, minerais, árvores, estruturas e decoração.
- Usar seed global e offsets por dimensão para reproduzir a geração.
- Gerar malhas apenas para faces expostas; agrupar por material/transparência e usar greedy meshing como otimização futura.
- Manter fronteiras entre chunks corretas: ao alterar bloco na borda, invalidar também o chunk vizinho.

4.2 Biomas e geração

O gerador deve produzir biomas por mapas de temperatura, umidade, elevação e ruído. A lista inicial deve contemplar variantes equivalentes às famílias clássicas: planícies, floresta, taiga, deserto, savana, selva, pântano, montanhas, oceano, rios, tundra/neve, mesa e ilhas do End. Nether e End possuem geradores próprios.

- Distribuir minérios por faixas de altura e regras por dimensão.
- Gerar cavernas, ravinas, lagos, rios, árvores, mineshafts, vilas, templos, fortalezas e fortalezas do Nether conforme regras de raridade.
- Permitir desligar estruturas e definir seed, tamanho do mundo e opções de geração.
- Separar algoritmo e parâmetros para balanceamento sem alteração de código.

4.3 Blocos e interação

- Quebrar blocos por ferramenta, resistência, estado do jogador e regras do bloco.
- Colocar blocos em faces válidas, respeitando colisão, orientação, estado e inventário.
- Suportar blocos com estados: orientação, metade, estágio, ativação, nível de líquido e variações.
- Implementar seleção por raycast a partir do centro da câmera e feedback visual de bloco-alvo.
- Controlar atualização de vizinhos, gravidade de blocos específicos, propagação de fluidos e atualizações de redstone.

4.4 Inventário, itens e crafting

- Inventário do jogador com hotbar, slots principais, armadura, mão secundária e cursor de arraste.
- Empilhamento limitado por item; itens não empilháveis devem ter durabilidade/metadados próprios.
- Receitas em grade 2×2 e 3×3, receitas de forno e receitas especiais.
- Crafting deve validar ingredientes, quantidades, padrão, resultado e sobra de recipientes.
- Fornalha com entrada, combustível, saída, progresso e consumo de combustível.
- Baús, fornalhas, dispensers, droppers, funis e outros containers com inventário próprio.

4.5 Sobrevivência e jogador

- Vida, fome, saturação, regeneração, dano, armadura, resistência, experiência e níveis.

- Movimentação: andar, correr, saltar, agachar, nadar, escalar quando aplicável e voar no Criativo.
- Queda, fogo, sufocamento, afogamento, lava, explosões, projéteis e efeitos de status.
- Morte: soltar itens conforme regra, registrar local, exibir tela de morte e permitir respawn.
- Descanso em cama para definir ponto de respawn e avançar o tempo quando as regras permitirem.

4.6 Entidades e IA

Implementar entidades com ciclo de vida, atributos, colisores, navegação, percepção, alvo, comportamento e drops. A IA deve ser orientada a estados/árvores de comportamento simples e atualizada em frequência inferior à renderização.

- Passivos: animais de criação e criaturas ambientais; reprodução, alimentação e drops.
- Neutros: criaturas que só atacam sob condições específicas.
- Hostis: zumbis, esqueletos, aranhas, creepers, endermen, bruxas, slimes e variantes relevantes.
- Especiais: aldeões, golems, guardiões, shulkers, dragão do End e entidades do Nether.
- Spawn controlado por bioma, luz, distância, dificuldade, limite por região e regras de dimensão.

4.7 Combate e equipamentos — marco 1.9

- Armas corpo a corpo com dano, alcance, durabilidade e velocidade de ataque.
- Tempo de recarga de ataque e indicador de cooldown; ataques antes da recarga causam dano reduzido.
- Escudo equipável na mão secundária, bloqueio direcional e interação com ataques/projéteis.
- Arco, flechas comuns, espectrais e com ponta; poções arremessáveis e persistentes.
- Armaduras, encantamentos defensivos/ofensivos e efeitos de status.
- Machados podem atuar como arma com perfil próprio; balanceamento deve ser configurável.

4.8 Dimensões e progressão

Dimensão	Requisitos
Mundo principal	Terreno, biomas, estruturas, clima, ciclo dia/noite e respawn
Nether	Portal, terreno próprio, lava, fortalezas, mobs e recursos exclusivos
End	Ilhas, portal, cristais, dragão, saída/portal de retorno e progressão final
End externo	Portais de acesso após o dragão, ilhas distantes, cidades do End, navios e shulkers

- Implementar elytra, foguetes não são necessários no escopo 1.9; voo deve respeitar física e colisões.
- Implementar chorus fruit, chorus plant, dragon breath, lingering potions, tipped arrows, spectral arrows e shulker shells.
- Implementar encantamentos Mending e Frost Walker, além dos encantamentos-base anteriores.

4.9 Redstone e mecanismos

Criar uma rede de atualização de sinais com níveis discretos, propagação, prioridade e prevenção de ciclos infinitos. A simulação deve ser separada da malha visual.

- Componentes-base: redstone dust, torch, repeater, comparator, lever, button, pressure plate, tripwire, daylight sensor.
- Pistons e sticky pistons, doors, trapdoors, gates, dispensers, droppers, hoppers e observers não fazem parte do escopo-base 1.9 (observer é posterior).
- Blocos musicais e mecanismos de interação devem emitir eventos para áudio.

5. Requisitos funcionais

ID	Requisito	Prioridade
RF-001	O sistema deve permitir criar um mundo com nome, seed, modo de jogo, dificuldade e opções de geração.	Obrigatório
RF-002	O sistema deve listar, abrir, renomear e excluir mundos locais, com confirmação de exclusão.	Obrigatório
RF-003	O mundo deve ser gerado proceduralmente a partir de seed e parâmetros definidos.	Obrigatório
RF-004	O mundo deve carregar e descarregar chunks conforme a posição do jogador e distância de renderização.	Obrigatório
RF-005	O jogador deve conseguir quebrar e colocar blocos respeitando alcance, ferramenta, estado e inventário.	Obrigatório
RF-006	O sistema deve persistir alterações de blocos, entidades relevantes, inventários e estado do mundo.	Obrigatório
RF-007	O jogador deve possuir inventário, hotbar, slots de armadura e slot de mão secundária.	Obrigatório
RF-008	O sistema deve suportar coleta, empilhamento, divisão, movimentação e descarte de itens.	Obrigatório
RF-009	O sistema deve validar receitas e produzir itens em crafting 2×2, 3×3 e fornalha.	Obrigatório
RF-010	O jogador deve possuir vida, fome, dano, armadura, experiência e efeitos de status no modo Sobrevivência.	Obrigatório
RF-011	O sistema deve implementar ciclo dia/noite, clima e regras de spawn.	Obrigatório
RF-012	O sistema deve implementar entidades com atributos, IA, colisão, drops e regras de spawn.	Obrigatório
RF-013	O sistema deve suportar combate corpo a corpo, arco, escudo e cooldown de ataque.	Obrigatório
RF-014	O sistema deve implementar encantamentos, poções e efeitos compatíveis com o limite 1.9.	Obrigatório
RF-015	O sistema deve permitir acessar Nether e End por portais e cumprir a progressão do dragão.	Obrigatório
RF-016	O sistema deve implementar mecanismos e circuitos básicos de redstone do escopo definido.	Obrigatório
RF-017	O modo Criativo deve permitir voo, invulnerabilidade e acesso ao catálogo de itens/blocos.	Obrigatório
RF-018	O jogador deve poder morrer e reaparecer conforme cama/ponto de spawn e regras do modo.	Obrigatório
RF-019	O sistema deve oferecer pausa, configurações de áudio, vídeo, controles e acessibilidade básica.	Obrigatório
RF-020	O sistema deve salvar automaticamente em intervalos configuráveis e ao encerrar de forma segura.	Obrigatório
RF-021	O sistema deve permitir mapear teclas e botões de mouse para ações do jogo.	Obrigatório
RF-022	O sistema deve mostrar HUD com mira, hotbar, vida, fome, armadura, experiência e indicadores contextuais.	Obrigatório
RF-023	O sistema deve permitir alterar distância de renderização e qualidade gráfica dentro de limites seguros.	Obrigatório
RF-024	O sistema deve oferecer ferramentas de diagnóstico para FPS, chunks, memória e tempo de geração.	Desejável
RF-025	O sistema poderá oferecer espectador, hardcore, presets de mundo e comandos locais de administração.	Opcional
RF-026	O sistema poderá permitir pacotes de recursos locais e configuração de receitas/loot por arquivos de dados.	Opcional
RF-027	O sistema poderá incluir modo LAN ou multiplayer futuro, sem exigir isso no primeiro lançamento.	Futuro

6. Requisitos não funcionais

ID	Categoria	Critério
RNF-001	Desempenho	Meta inicial de 60 FPS em configuração média, com queda controlada durante geração; definir hardware mínimo após protótipo.
RNF-002	Tempo de resposta	Entrada do jogador deve ser refletida imediatamente; interação comum deve responder em até 100 ms, fora de geração/carregamento.
RNF-003	Memória	Chunks fora da distância ativa devem ser descarregados; evitar retenção de geometrias e texturas sem referência.
RNF-004	Persistência	Salvar de forma atômica, com backup/arquivo temporário e validação de versão.
RNF-005	Recuperação	Falha durante gravação não deve corromper todos os dados do mundo; manter último snapshot válido.
RNF-006	Segurança	Renderer sem Node integration; IPC validado; filesystem acessível somente por API controlada.
RNF-007	Manutenibilidade	Domínio independente de UI e plataforma; módulos testáveis isoladamente.
RNF-008	Extensibilidade	Blocos, itens, receitas, entidades, biomas e loot devem ser definidos por registros/dados sempre que possível.
RNF-009	Compatibilidade	Suportar resoluções desktop comuns e escala de interface; validar Windows como plataforma inicial.
RNF-010	Usabilidade	Menus e inventários devem ser navegáveis por mouse e teclado; controles remapeáveis.
RNF-011	Acessibilidade	Opções de escala de UI, alternância de efeitos visuais, volume independente e remapeamento.
RNF-012	Observabilidade	Logs por subsistema, níveis de log e relatório de erro sem dados pessoais.
RNF-013	Testabilidade	Geradores, receitas, inventário, dano, salvamento e regras devem ter testes automatizados.
RNF-014	Privacidade	Sem telemetria ou coleta externa por padrão; qualquer envio futuro deve ser explícito e documentado.
RNF-015	Distribuição	Build reproduzível, instalador versionado e migração de save documentada.

7. Requisitos opcionais e evolução

ID	Funcionalidade	Observação
OP-001	Multiplayer LAN	Adicionar autoridade de mundo, protocolo, replicação e validação de ações.
OP-002	Servidor dedicado	Separar servidor headless da camada de renderização.
OP-003	Pacotes de conteúdo	Carregar definições locais de blocos, itens, receitas, loot e sons.
OP-004	Sistema de comandos	Console local para teleporte, hora, clima, modo de jogo e inspeção.
OP-005	Mapas e waypoints	Mapa explorado persistente e marcadores locais.
OP-006	Replay e screenshots	Captura de tela e replay de eventos para depuração.
OP-007	Modding API	Hooks controlados, registro de conteúdo e sandbox para scripts.
OP-008	Suporte a controle	Gamepad, navegação de UI e perfis de input.
OP-009	Linux/macOS	Empacotamento, permissões e testes por plataforma.
OP-010	Editor de mundo	Ferramentas de seleção, preenchimento e edição em lote.

8. Modelo de dados

8.1 Entidades principais

Entidade	Campos essenciais
World	id, name, seed, gameMode, difficulty, version, createdAt, lastPlayedAt, rules, dimensions
Dimension	id, type, generatorId, time, weather, spawnPoint, chunkIndex
Chunk	coord, blockPalette, blockData, lightData, entities, tileEntities, dirty, lastSaved
BlockDefinition	id, name, material, hardness, tool, drops, collision, renderLayer, states, tags
ItemStack	itemId, count, durability, metadata, enchantments
ItemDefinition	id, name, maxStack, maxDurability, category, icon, behavior
Entity	id, type, position, rotation, velocity, health, attributes, aiState, inventory, persistentData
Recipe	id, type, pattern/ingredients, result, remainder, unlockRule
PlayerState	position, rotation, health, hunger, saturation, xp, inventory, armor, offhand, spawn
WorldEvent	type, tick, position, source, payload

8.2 Identificadores e dados

Usar IDs numéricos compactos para blocos durante a simulação e IDs textuais estáveis nos arquivos de definição. Não persistir referências de objetos JavaScript. Metadados variáveis devem ser serializados em formato versionado.

```
BlockDefinition { id: string; numericId: number; hardness: number; toolTags: string[]; renderLayer: 'opaque'|'cutout'|'translucent'; states: StateSchema; drops: DropRule[] }

ItemStack { itemId: string; count: number; damage?: number; data?: Record; enchantments?: Enchantment[] }
```

8.3 Exemplo de estrutura de mundo

`world/`
`world.json` — metadados, seed, regras e versão
`player.dat` — estado do jogador
`dimensions/overworld/regions/` — chunks agrupados em regiões
`dimensions/nether/`
`dimensions/end/`
`backups/` — snapshots rotativos

9. Persistência, versionamento e integridade

- Salvar chunks modificados (dirty) em lotes, evitando gravar o mundo inteiro após cada ação.
- Usar escrita temporária seguida de rename/commit atômico quando o sistema de arquivos permitir.
- Manter versão de formato no arquivo do mundo e migrações sequenciais entre versões.
- Separar dados regeneráveis (malhas, caches, iluminação derivada) dos dados autoritativos (blocos, entidades persistentes, inventários).
- Executar autosave periódico, save ao pausar/fechar e flush em transições de dimensão.
- Exibir aviso se um mundo foi criado por versão incompatível; nunca sobrescrever silenciosamente dados não reconhecidos.
- Manter pelo menos um backup rotativo por mundo e permitir restauração manual.

9.1 Estratégia de serialização

Para protótipo, JSON pode armazenar metadados e configurações; para chunks, usar formato binário compacto com palette de blocos, índices e compressão. A escolha final deve ser validada por benchmark. Não serializar objetos Three.js, instâncias de física ou stores Zustand diretamente.

9.2 Salvamento concorrente

Cada mundo deve possuir uma fila de gravação. Uma versão/contador de alteração evita que uma gravação antiga sobrescreva uma mais nova. Ao fechar a janela, bloquear novas alterações, concluir ou cancelar com segurança as tarefas pendentes e só então encerrar.

10. Estrutura de pastas recomendada

```
src/ app/ # bootstrap React, router e providers features/ main-menu/ world-selection/ world-creation/ settings/ hud/ inventory/ crafting/ pause/ game/ core/ # GameLoop, Clock, EventBus, GameSession domain/ # World, Chunk, Block, Item, Entity, Player systems/ # input, movement, combat, survival, crafting worldgen/ # noise, biomes, structures, ores, dimensions rendering/ # Three.js, materials, chunk mesher, camera physics/ # Rapier adapters e colisões audio/ persistence/ content/ # definições JSON/TS de itens, blocos e receitas stores/ # Zustand: UI, settings, session snapshots shared/ # tipos, utilitários e componentes electron/ main/ preload/ workers/ # geração, meshing e tarefas pesadas tests/
```

A estrutura é uma sugestão. Evite importar componentes React dentro de game/domain. O domínio deve poder rodar em testes sem DOM, Electron ou WebGL.

11. Estado e navegação

11.1 Zustand

Zustand deve guardar estado de interface e snapshots pequenos do estado de sessão. Não é adequado armazenar todos os blocos do mundo, geometrias, milhares de entidades ou buffers grandes em uma store reativa.

Store	Responsabilidade
useUIStore	Tela ativa, overlays, menus, modal, foco e notificações
useSettingsStore	Áudio, vídeo, controles, acessibilidade e preferências persistidas
useSessionStore	ID do mundo atual, modo, dimensão, estado de pausa e indicadores resumidos
useInventoryUIStore	Janela aberta, slot selecionado, item arrastado e foco de navegação
GameSession (fora do Zustand)	Estado autoritativo do mundo, jogador, entidades, ticks e sistemas

11.2 React Router

Rotas sugeridas: / (menu), /worlds, /worlds/new, /settings, /credits e /game/:worldId. A rota do jogo deve montar o canvas e a UI de jogo; ao sair, executar o fluxo de pausa/salvamento e desmontagem da sessão.

11.3 Comunicação com o motor

- UI envia comandos (ex.: placeBlock, selectSlot, setPause) por uma fachada de aplicação.
- Motor publica eventos/snapshots de baixa frequência para atualizar HUD e interface.
- Não disparar atualização React por bloco ou por frame.
- Usar selectors Zustand e memoização para limitar renders.

12. Controles e interface

Ação	Controle padrão sugerido
Mover	W, A, S, D
Pular	Espaço
Agachar	Shift
Correr	Ctrl ou toque duplo em direção
Quebrar/atacar	Botão esquerdo do mouse
Colocar/usar	Botão direito do mouse
Selecionar hotbar	1–9 / roda do mouse
Inventário	E
Soltar item	Q
Trocar mão secundária	F
Pausar	Esc

Os controles acima são defaults configuráveis, não valores fixos. A UI deve suportar redimensionamento, escala, foco correto, tooltip, arraste de slots, atalhos e feedback de ação inválida.

12.1 Telas mínimas

- Menu principal: Jogar, Configurações, Créditos e Sair.
- Seleção de mundo: lista, criar, editar opções, excluir e abrir.
- Criação de mundo: nome, seed, modo, dificuldade, estruturas e opções avançadas.
- HUD: mira, hotbar, vida, fome, armadura, experiência e status.
- Inventário: jogador, crafting, armadura, mão secundária e containers.
- Pause: voltar ao jogo, configurações, salvar/sair para menu.
- Configurações: vídeo, áudio, controles, interface e acessibilidade.
- Telas contextuais: fornalha, baú, crafting, morte e créditos.

13. Segurança e limites de confiança

- Renderer não deve acessar fs, process, shell ou APIs Node diretamente.
- Preload deve expor métodos específicos, por exemplo listWorlds(), readWorldMetadata(), saveWorldChunk() e showOpenDialog().
- Validar tipo, tamanho, caminho e versão de todo payload de IPC.
- Normalizar caminhos e impedir path traversal; restringir operações à pasta de dados do jogo.
- Não executar arquivos de conteúdo como código. Conteúdo externo deve ser dados declarativos validados.
- Tratar saves como dados potencialmente corrompidos: validar limites, IDs desconhecidos e tamanhos antes de carregar.
- Evitar eval, Function e carregamento remoto de scripts.

14. Desempenho e otimização

- Frustum culling, distance culling e occlusion/culling adicional quando justificável.

- Instancing por tipo de bloco/objeto e atlas de texturas para reduzir draw calls.
- Greedy meshing e geração incremental de malhas por chunk.
- Separar chunks opacos, cutout e transparentes; ordenar transparências quando necessário.
- Reutilizar buffers e objetos; evitar alocação de vetores e arrays em loops críticos.
- Limitar entidades ativas e frequência de IA por distância.
- Usar LOD ou redução de detalhes para regiões distantes, se necessário.
- Medir CPU, GPU, memória, draw calls, chunks carregados, tempo de meshing e tempo de save.

14.1 Metas de protótipo

Métrica	Meta inicial de engenharia
Frame time	16,7 ms para 60 FPS em cenário de referência, após otimização
Simulação	Tick estável sem acumular atraso indefinidamente
Carregamento	Chunks próximos priorizados; evitar congelamento perceptível da thread de UI
Save	Salvar apenas dados alterados e manter progresso recuperável
Uso de memória	Definir baseline em máquina mínima após primeiro benchmark real

As metas são objetivos de validação, não garantias. O hardware mínimo e os limites de distância devem ser definidos com protótipos e medições reais.

15. Testes e critérios de aceitação

Nível	O que testar	Exemplos
Unitário	Regras puras do domínio	Receitas, empilhamento, dano, drops, seed, tabelas de loot
Integração	Comunicação entre sistemas	Quebrar bloco atualiza inventário, chunk e save; portal troca dimensão
Sistema	Fluxos completos	Criar mundo, jogar, salvar, fechar, reabrir e continuar
Performance	Cenários pesados	Explosões, grandes construções, entidades, chunks e redstone
Compatibilidade	SO e hardware	Inicialização, resolução, input, áudio e caminhos de save
Segurança	IPC e arquivos	Payload inválido, caminho externo, save corrompido e conteúdo malformado

15.1 Critérios de aceite do MVP

- O jogador consegue criar um mundo, entrar, mover-se, olhar ao redor, minerar e colocar blocos.
- Chunks são gerados proceduralmente e carregados ao redor do jogador sem reconstruir o mundo inteiro.
- Inventário, hotbar, crafting básico e fornalha funcionam.
- Sobrevivência inclui vida, fome, dano, morte e respawn.
- Um conjunto inicial de mobs hostis e passivos funciona com spawn e drops.
- O mundo pode ser salvo, fechado e carregado preservando alterações.
- Menu, pausa, configurações essenciais e controles remapeáveis estão funcionais.
- Não há dependência de internet para iniciar e jogar single-player.

16. Plano de desenvolvimento por fases

Fase	Entregáveis	Saída esperada
0 — Fundação	Electron seguro, Vite, React, Tailwind, Router, logging e build	Aplicação desktop abre e navega
1 — Protótipo voxel	Three.js, câmera FPS, raycast, blocos, chunk e meshing	Andar, olhar, quebrar e colocar
2 — Mundo	Seed, terreno, biomas iniciais, cavernas, árvores e streaming	Mundo procedural explorável
3 — Sobrevivência	Jogador, inventário, itens, crafting, forno, fome e vida	Loop básico de sobrevivência
4 — Entidades	Mobs, IA, spawn, drops, combate e dificuldade	Ecossistema e ameaças
5 — Conteúdo 1.9	Nether, End, redstone, poções, encantamentos, escudo, elytra	Paridade funcional de escopo ampliado
6 — Persistência	Formato final, autosave, backup e migração	Mundos persistentes confiáveis
7 — Polimento	Áudio, UI, otimização, acessibilidade, testes e instalador	Release single-player

16.1 Estratégia recomendada

Não tente implementar todo o catálogo de blocos e mobs antes de validar o motor. Primeiro construa um vertical slice pequeno: um bioma, alguns blocos, inventário, crafting, um mob, save/load e geração de chunks. Só então amplie conteúdo e dimensões. Isso reduz o risco de descobrir tarde problemas de arquitetura, desempenho ou persistência.

17. Riscos técnicos e decisões em aberto

Risco/decisão	Impacto	Mitigação
Escopo equivalente a toda a 1.9	Muito alto: grande volume de conteúdo e regras	Definir milestones e catálogo priorizado; não prometer paridade total no MVP
Renderização 3D ausente na stack original	Crítico: React não substitui engine 3D	Adicionar Three.js ou Babylon.js; este SDD assume Three.js
Geração e meshing no thread principal	Alto: travamentos	Workers, filas e limites por frame
Transparência e líquidos	Médio/alto: artefatos visuais	Camadas de renderização e testes de ordenação
Redstone e atualizações em cadeia	Alto: loops e picos de CPU	Fila de eventos, limite de propagação e testes de circuito
Save corrompido	Crítico: perda de progresso	Gravação atômica, backup, validação e migração
Compatibilidade de assets	Alto: questões legais e técnicas	Criar assets próprios; não importar arquivos proprietários
Dependências e versões	Médio: breaking changes	Fixar versões, lockfile e atualização controlada

17.1 Decisões que precisam ser fechadas antes da produção

- Dimensão vertical e formato de chunk: altura fixa ou seções verticais.
- Alvo de hardware mínimo e distância padrão de renderização.
- Formato binário definitivo para chunks e compressão.
- Biblioteca de física e estratégia de colisão do jogador.
- Escopo exato de paridade: todas as mecânicas 1.9 ou seleção de recursos essenciais.

- Suporte inicial somente Windows ou Windows + Linux.
- Política de conteúdo: assets autorais, licença e atribuição de bibliotecas.

18. Glossário

Termo	Definição
Voxel/bloco	Unidade cúbica básica que compõe o terreno e construções.
Chunk	Região do mundo processada e renderizada como unidade.
Meshing	Conversão de dados de blocos em geometria renderizável.
Seed	Valor que determina a sequência de geração procedural.
Tick	Passo discreto da simulação do jogo.
Greedy meshing	Técnica que combina faces adjacentes para reduzir polígonos.
Offhand	Slot de mão secundária usado, por exemplo, para escudo.
Dirty chunk	Chunk alterado desde o último salvamento.
IPC	Comunicação entre processos do Electron.

19. Referências de implementação

- Documentação oficial do Electron: arquitetura main/renderer, preload, context isolation e segurança.
- Documentação oficial do Vite: configuração de build e integração com Electron.
- Documentação oficial do React e React Router: composição de UI e navegação.
- Documentação oficial do Zustand: stores, selectors e persistência de preferências.
- Documentação oficial do Three.js: WebGLRenderer, BufferGeometry, InstancedMesh, Raycaster e materiais.
- Documentação oficial do Rapier: corpos rígidos, colliders e consultas espaciais.

As referências acima são pontos de partida técnicos. Antes de iniciar a implementação, fixe versões compatíveis das dependências e valide a licença de cada biblioteca.

20. Conclusão

Este SDD define uma arquitetura desktop single-player para um sandbox voxel com escopo de conteúdo até a referência 1.9. A stack solicitada atende à interface e à aplicação desktop, mas precisa de uma camada de renderização 3D dedicada — recomendada aqui como Three.js — e de uma solução de física para o jogador e entidades.

A recomendação central é construir primeiro o núcleo técnico (chunks, meshing, input, mundo, persistência) e validar um ciclo jogável mínimo antes de expandir para todas as dimensões, entidades, redstone e recursos avançados. O documento deve ser atualizado conforme decisões de desempenho, formato de save e paridade de funcionalidades forem validadas.